UNIVERSIDAD TECNOLÓGICA DE LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

TEMA:

Sistemas de Graficación en Computación Gráfica

Estudiante:

Ortiz Chaccara Vladimir Alexandre

Docente:

Mg. Elvio Tintaya Zegarra

ABANCAY – APURÍMAC – PERÚ

30 de diciembre 2025

¿Qué es un Sistema de Graficación?

Un Sistema de Graficación combina hardware y software para generar, procesar y mostrar imágenes y modelos en una computadora. Es la infraestructura que permite desde una simple visualización 2D hasta escenas 3D interactivas en tiempo real. En este apartado describimos sus dos componentes esenciales y su papel en la cadena de producción gráfica.

Hardware de Graficación

La columna vertebral física: GPUs, controladores de vídeo, VRAM y dispositivos de entrada y salida. Aquí ocurren los cálculos masivos y el transporte de la información hacia el monitor o dispositivo final.

Software de Graficación

Bibliotecas y motores que permiten emitir comandos a la GPU. Incluye APIs como OpenGL, motores (Unity, Unreal) y aplicaciones como Blender para modelado y procesamiento.

Componentes críticos del hardware:

GPU

Procesador paralelo optimizado para operaciones matriciales y sombreado, esencial para rasterización y cálculo de vértices y fragmentos.

VRAM y Controlador de Vídeo

Almacenan buffers (frame buffer, depth buffer) y gestionan la transferencia de datos hacia la salida de vídeo.

Dispositivos de E/S

Monitores (LED, OLED), impresoras 3D y cascos VR para salida; mouse, tabletas y sensores para entrada.

Capas del software:

APIs Gráficas

OpenGL, Vulkan o Direct3D: abstracciones que permiten al software controlar la GPU de forma portátil.

Motores Gráficos

Frameworks de alto nivel que gestionan escena, físicas y renderizado para acelerar el desarrollo.

Software de Aplicación

Herramientas de creación (p. ej. Blender) y utilidades de edición que generan los activos empleables por el motor.

Conceptos Clave del Sistema

Comprender la terminología fundamental es imprescindible para seguir el flujo de trabajo gráfico. A continuación, se explican los conceptos básicos y su relevancia práctica dentro del pipeline gráfico:

Píxel

Unidad mínima de representación de color en una imagen digital. Cada píxel almacena información de color (por ejemplo, RGB o RGBA) que, en conjunto con otros píxeles, forma la imagen visible en pantalla.

Resolución

Medida en ancho × alto (píxeles). Determina la densidad de información visual y afecta directamente al rendimiento: mayor resolución → más píxeles que rasterizar por fotograma.

Buffer de Trama

Área de memoria (frecuentemente en VRAM) donde se construye la imagen final antes de enviarla al monitor. Incluye color buffer y depth/stencil buffers que permiten composiciones correctas y pruebas de profundidad.

Pipeline Gráfico

Conjunto de etapas que transforman datos 3D en una imagen 2D. Incluye: procesamiento de vértices, montaje de primitivas, rasterización, procesamiento de fragmentos (shading) y pruebas finales (depth test, blending).

Etapas relevantes (resumen)

Vertex Processing: transformación de coordenadas y cálculo de atributos.

Clipping & Culling: eliminación de geometría fuera de cámara o no visible.

Rasterización: conversión de primitivas en fragmentos de píxeles.

Fragment Shading: cálculo final de color por píxel (texturas, iluminación).

Implicaciones para desarrolladores

Optimizar cargas de vértices, reducir overdraw y gestionar eficientemente buffers y texturas son habilidades prácticas que impactan en rendimiento y calidad visual.

El diagrama resume la transformación esencial: primero se procesan vértices y matrices, luego la rasterización convierte primitivas en fragmentos y finalmente, el sombreado calcula el color final de cada píxel.

Consejo práctico

Perfilamiento: emplea herramientas (profiler GPU/CPU) para identificar cuellos de botella; prioriza reducir pasos costosos en el shader y optimizar transferencias a la VRAM.

Lecturas recomendadas

- Documentación oficial de OpenGL
- Tutoriales sobre pipeline gráfico
- Proyectos prácticos en Blender y motores en tiempo real.

Recomendación:

Experimenta con OpenGL para entender la relación directa entre llamadas de la API y operaciones en la GPU; usa Blender para crear activos 3D y probar pipelines simples.